

UART

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
```

```
ENTITY UART IS
```

```
    PORaT (
        RxD, Load, GReset, GClock : IN STD_LOGIC;
        State_Information : IN STD_LOGIC_VECTOR(5 downto 0);
        TxD, CanProceedState : OUT STD_LOGIC
    );
END UART;
```

```
ARCHITECTURE struct of UART IS
```

```
    COMPONENT UARTTransmitter
```

```
        PORT (
            loadTDR, GReset, GClock, BaudRate, shiftLoad : IN STD_LOGIC;
            D_In : IN STD_LOGIC_VECTOR(6 downto 0);
            TxD, TDRE, countEqualTenOut : OUT STD_LOGIC
        );
    END COMPONENT;
```

```
    COMPONENT counter_4bit
```

```
        PORT (
            CLK : in STD_LOGIC;
i_enable : in STD_LOGIC;
            i_hold : IN STD_LOGIC;
            i_reset : IN STD_LOGIC;
COUNT : out STD_LOGIC_VECTOR (3 downto 0)
        );
    END COMPONENT;
```

```
    COMPONENT equality_Comparator_4bit
```

```
        PORT (
            A, B : IN STD_LOGIC_VECTOR(3 downto 0);
            isEqual : OUT STD_LOGIC
        );
    END COMPONENT;
```

```
    COMPONENT UARTReciever
```

```
        PORT (
            RxD, GReset, BClkx8, GClock : IN STD_LOGIC;
            RecieveOut : OUT STD_LOGIC_VECTOR(7 downto 0);
```

```

        RDRF : OUT STD_LOGIC
    );
END COMPONENT;

```

```

COMPONENT dFF_8bit
    PORT (
        i_d      : IN  STD_LOGIC_VECTOR(7 downto 0);
        i_en      : IN  STD_LOGIC;
        i_reset   : IN  STD_LOGIC;
        i_clock   : IN  STD_LOGIC;
        o_q       : OUT STD_LOGIC_VECTOR(7 downto 0);
        o_qBar    : OUT STD_LOGIC_VECTOR(7 downto 0)
    );
END COMPONENT;

```

```

COMPONENT BaudRateGen
    PORT (
        GClock, GReset : IN  STD_LOGIC;
        SEL : IN  STD_LOGIC_VECTOR(2 downto 0);
        BClk, BClkx8 : OUT STD_LOGIC
    );
END COMPONENT;

```

```

COMPONENT InterruptGenerator
    PORT (
        TIE, RIE, RDRF, OE, TDRE: in std_logic;
        IRQ : out std_logic_vector(1 downto 0)
    );
END COMPONENT;

```

```

COMPONENT UARTAddressDecode
    PORT (
        Address : in  STD_LOGIC_VECTOR(1 downto 0);
        RWbar   : in  STD_LOGIC;
        DecodeOut : out STD_LOGIC_VECTOR(5 downto 0)
    );
END COMPONENT;

```

```

COMPONENT d_FF
PORT(
    i_d, i_en, i_reset : IN  STD_LOGIC;
    i_clock : IN  STD_LOGIC;
    o_q     : OUT STD_LOGIC;
    o_qBar  : OUT STD_LOGIC

```

```
);
END COMPONENT;
```

```
COMPONENT mux_8to1_1bit
PORT (
    d_in : IN STD_LOGIC_VECTOR(7 downto 0);
    sel : IN STD_LOGIC_VECTOR(2 downto 0);
    d_out : OUT STD_LOGIC
);
END COMPONENT;
```

```
SIGNAL TransitorySCSROutLatched, TransitorySCCRLatched, DataBus,
TransitoryRDROut, TransitorySCCROut, TransitorySCSROut, TransitoryRDRHeld :
STD_LOGIC_VECTOR(7 downto 0);
SIGNAL TransitoryMSTLState, TransitorySSTLState : STD_LOGIC_VECTOR(1 downto
0);
SIGNAL selectBits, ByteToSelect : STD_LOGIC_VECTOR(2 downto 0);
SIGNAL BytesSent : STD_LOGIC_VECTOR(3 downto 0);
SIGNAL TransitoryTxD, TransitoryCanProceedStateLatched, transitoryCountEqualTen,
incrementByteCounter, ResetByteCounter, TransitoryCanProceedState, DoneSendingWord,
BClk, BClkx8, transitoryRDRF, transitoryTDRE, transitoryTDRELatched : STD_LOGIC;
SIGNAL ByteToSend, transitoryTDRByte, ByteCR, ByteM, ByteY, ByteR,
ByteUnderscore, ByteS, ByteG : STD_LOGIC_VECTOR(6 downto 0);
SIGNAL WordToSend, State1Word, State2Word, State3Word, State4Word :
STD_LOGIC_VECTOR(41 downto 0);
begin
    -- 7 bit encoding format
    ByteM <= "1001101";
    ByteY <= "1111001"; --lowercase y
    ByteR <= "1110010"; --lowercase r
    ByteUnderscore <= "1011111";
    ByteS <= "1010011";
    ByteG <= "1100111"; --lowercase g
    ByteCR <= "0001101";

    State1Word <= ByteM & ByteG & ByteUnderscore & ByteS & ByteR & ByteCR;
    State2Word <= ByteM & ByteY & ByteUnderscore & ByteS & ByteR & ByteCR;
    State3Word <= ByteM & ByteR & ByteUnderscore & ByteS & ByteG & ByteCR;
    State4Word <= ByteM & ByteR & ByteUnderscore & ByteS & ByteY & ByteCR;
    incrementByteCounter <= transitoryTDRELatched AND TransitoryCountEqualTen;
    d_FF_proceedStateLatched : d_FF
    PORT MAP (
        i_d => TransitoryCanProceedState,
        i_en => '1',
```

```

        i_reset => GReset,
        i_clock => GClock,
        o_q => TransitoryCanProceedStateLatched,
        o_qBar => open
    );
CanProceedState <= TransitoryCanProceedStateLatched;
ResetByteCounter <= GReset OR TransitoryCanProceedStateLatched;

counter_4bit_BytesSent : counter_4bit
    PORT MAP (
        CLK => incrementByteCounter,
        i_enable => '1',
        i_hold => '0',
        i_reset => ResetByteCounter,
        COUNT => BytesSent
    );

equality_Comparator_4bit_sent6Bytes : equality_Comparator_4bit
    PORT MAP (
        A => BytesSent,
        B => "0110",
        isEqual => TransitoryCanProceedState
    );

dFF_8bit_SCCR : dFF_8bit
    PORT MAP (
        i_d => TransitorySCCROut,
        i_en => '1',
        i_reset => GReset,
        i_clock => GClock,
        o_q => TransitorySCCRLatched,
        o_qBar => open
    );
TransitorySCSROut(6) <= transitoryRDRF;

dFF_8bit_SCSR : dFF_8bit
    PORT MAP (
        i_d => TransitorySCSROut,
        i_en => '1',
        i_reset => GReset,
        i_clock => GClock,
        o_q => TransitorySCSROutLatched,
        o_qBar => open
    );

```

```
selectBits <= "000";
```

```
BaudRateGen_inst : BaudRateGen
```

```
  PORT MAP (  
    GClock => GClock,  
    GReset => GReset,  
    SEL => selectBits,  
    BClk => BClk,  
    BClkx8 => BClkx8  
  );
```

```
UARTReciever_inst : UARTReciever
```

```
  PORT MAP (  
    RxD => TransitoryTxD,  
    GClock => GClock,  
    BClkx8 => BClkx8,  
    GReset => GReset,  
    RecieveOut => TransitoryRDROut,  
    RDRF => transitoryRDRF  
  );
```

```
dFF_8bit_HoldRecieverOut : dFF_8bit
```

```
  PORT MAP (  
    i_d => TransitoryRDROut,  
    i_en => '1',  
    i_reset => GReset,  
    i_clock => transitoryRDRF,  
    o_q => TransitoryRDRHeld,  
    o_qBar => open  
  );
```

```
UARTTransmitter_inst : UARTTransmitter
```

```
  PORT MAP (  
    loadTDR => '1',  
    GReset => GReset,  
    GClock => GClock,  
    BaudRate => BClk,  
    shiftLoad => transitoryTDRELatched,  
    D_In => ByteToSend,  
    TxD => TransitoryTxD,  
    TDRE => transitoryTDRE,  
    countEqualTenOut => TransitoryCountEqualTen
```

```

    );

    d_FF_latchTDRE : d_FF
    PORT MAP (
        i_d => transitoryTDRE,
        i_en => '1',
        i_reset => GReset,
        i_clock => GClock,
        o_q => transitoryTDRELatched,
        o_qBar => open
    );

    TxD <= TransitoryTxD;

```

```

end struct;

```

SerialInParallelOut

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY SerialInParallelOut_8Bit IS
    PORT(
        i_reset : IN    STD_LOGIC;
        i_enable : IN    STD_LOGIC;
        i_shift : IN    STD_LOGIC;
        i_clock : IN    STD_LOGIC;
        s_out : OUT STD_LOGIC_VECTOR(7 downto 0));
END SerialInParallelOut_8Bit;

```

```

ARCHITECTURE rtl OF SerialInParallelOut_8Bit IS
    SIGNAL d_sigs : STD_LOGIC_VECTOR(7 downto 0);
    SIGNAL q_outs : STD_LOGIC_VECTOR(7 downto 0);

    COMPONENT d_FF IS
        PORT(
            i_d, i_en, i_reset : IN    STD_LOGIC;
            i_clock : IN    STD_LOGIC;
            o_q, o_qBar : OUT STD_LOGIC);
    END COMPONENT;

```

```

BEGIN

```

```

    bit0: d_FF
        Port map(

```

```
        i_reset => i_reset,  
        i_en => i_enable,  
        i_d => q_outs(1),  
        i_clock => i_clock,  
        o_q => q_outs(0)  
    );
```

bit1: d_FF

```
    Port map(  
        i_reset => i_reset,  
        i_en => i_enable,  
        i_d => q_outs(2),  
        i_clock => i_clock,  
        o_q => q_outs(1)  
    );
```

bit2: d_FF

```
    Port map(  
        i_reset => i_reset,  
        i_en => i_enable,  
        i_d => q_outs(3),  
        i_clock => i_clock,  
        o_q => q_outs(2)  
    );
```

bit3: d_FF

```
    Port map(  
        i_reset => i_reset,  
        i_en => i_enable,  
        i_d => q_outs(4),  
        i_clock => i_clock,  
        o_q => q_outs(3)  
    );
```

bit4: d_FF

```
    Port map(  
        i_reset => i_reset,  
        i_en => i_enable,  
        i_d => q_outs(5),  
        i_clock => i_clock,  
        o_q => q_outs(4)  
    );
```

bit5: d_FF

```

        Port map(
            i_reset => i_reset,
            i_en => i_enable,
            i_d => q_outs(6),
            i_clock => i_clock,
            o_q => q_outs(5)
        );

bit6: d_FF
    Port map(
        i_reset => i_reset,
        i_en => i_enable,
        i_d => q_outs(7),
        i_clock => i_clock,
        o_q => q_outs(6)
    );

bit7: d_FF
    Port map(
        i_reset => i_reset,
        i_en => i_enable,
        i_d => i_shift,
        i_clock => i_clock,
        o_q => q_outs(7)
    );

-- Output Driver
    s_out      <=    q_outs;

END rtl;

```

ParallelInSerialOut_8Bit

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY ParallelInSerialOut_8Bit IS
    PORT(
        i_d          : IN    STD_LOGIC_VECTOR(9 downto 0);
        i_reset      : IN    STD_LOGIC;
        i_enable      : IN    STD_LOGIC;
        i_shift       : IN    STD_LOGIC;
        i_shift_Load  : IN    STD_LOGIC;
        i_clock       : IN    STD_LOGIC;
        s_out         : OUT   STD_LOGIC);

```

```
END ParallelInSerialOut_8Bit;
```

```
ARCHITECTURE rtl OF ParallelInSerialOut_8Bit IS
```

```
    SIGNAL d_sigs : STD_LOGIC_VECTOR(9 downto 0);
```

```
    SIGNAL q_outs : STD_LOGIC_VECTOR(9 downto 0);
```

```
    COMPONENT d_FF IS
```

```
    PORT(
```

```
        i_d, i_en, i_reset          : IN    STD_LOGIC;
```

```
        i_clock                    : IN    STD_LOGIC;
```

```
        o_q, o_qBar                : OUT   STD_LOGIC);
```

```
    END COMPONENT;
```

```
BEGIN --
```

```
    d_sigs(0) <= ((i_d(0) AND i_shift_Load) OR (q_outs(1) AND not i_shift_Load));
```

```
    d_sigs(1) <= ((i_d(1) AND i_shift_Load) OR (q_outs(2) AND not i_shift_Load));
```

```
    d_sigs(2) <= ((i_d(2) AND i_shift_Load) OR (q_outs(3) AND not i_shift_Load));
```

```
    d_sigs(3) <= ((i_d(3) AND i_shift_Load) OR (q_outs(4) AND not i_shift_Load));
```

```
    d_sigs(4) <= ((i_d(4) AND i_shift_Load) OR (q_outs(5) AND not i_shift_Load));
```

```
    d_sigs(5) <= ((i_d(5) AND i_shift_Load) OR (q_outs(6) AND not i_shift_Load));
```

```
    d_sigs(6) <= ((i_d(6) AND i_shift_Load) OR (q_outs(7) AND not i_shift_Load));
```

```
    d_sigs(7) <= ((i_d(7) AND i_shift_Load) OR (q_outs(8) AND not i_shift_Load));
```

```
    d_sigs(8) <= ((i_d(8) AND i_shift_Load) OR (q_outs(9) AND not i_shift_Load));
```

```
    d_sigs(9) <= ((i_d(9) AND i_shift_Load) OR (i_shift AND not i_shift_Load));
```

```
bit0: d_FF
```

```
    Port map(
```

```
        i_reset => i_reset,
```

```
        i_en => i_enable,
```

```
        i_d => d_sigs(0),
```

```
        i_clock => i_clock,
```

```
        o_q => q_outs(0)
```

```
    );
```

```
bit1: d_FF
```

```
    Port map(
```

```
        i_reset => i_reset,
```

```
        i_en => i_enable,
```

```
        i_d => d_sigs(1),
```

```
        i_clock => i_clock,
```

```
        o_q => q_outs(1)
```

```
    );
```

```
bit2: d_FF
```

```
Port map(  
    i_reset => i_reset,  
    i_en => i_enable,  
    i_d => d_sigs(2),  
    i_clock => i_clock,  
    o_q => q_outs(2)  
);
```

bit3: d_FF

```
Port map(  
    i_reset => i_reset,  
    i_en => i_enable,  
    i_d => d_sigs(3),  
    i_clock => i_clock,  
    o_q => q_outs(3)  
);
```

bit4: d_FF

```
Port map(  
    i_reset => i_reset,  
    i_en => i_enable,  
    i_d => d_sigs(4),  
    i_clock => i_clock,  
    o_q => q_outs(4)  
);
```

bit5: d_FF

```
Port map(  
    i_reset => i_reset,  
    i_en => i_enable,  
    i_d => d_sigs(5),  
    i_clock => i_clock,  
    o_q => q_outs(5)  
);
```

bit6: d_FF

```
Port map(  
    i_reset => i_reset,  
    i_en => i_enable,  
    i_d => d_sigs(6),  
    i_clock => i_clock,  
    o_q => q_outs(6)  
);
```

```

bit7: d_FF
    Port map(
        i_reset => i_reset,
        i_en => i_enable,
        i_d => d_sigs(7),
        i_clock => i_clock,
        o_q => q_outs(7)
    );

bit8: d_FF
    Port map(
        i_reset => i_reset,
        i_en => i_enable,
        i_d => d_sigs(8),
        i_clock => i_clock,
        o_q => q_outs(8)
    );

bit9: d_FF
    Port map(
        i_reset => i_reset,
        i_en => i_enable,
        i_d => d_sigs(9),
        i_clock => i_clock,
        o_q => q_outs(9)
    );

-- Output Driver
    s_out      <=    q_outs(0);

END rtl;

```

InterruptGenerator

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity InterruptGenerator is
    Port (
        TIE, RIE, RDRF, OE, TDRE: in std_logic;
        IRQ: out std_logic_vector(1 downto 0)
    );
end InterruptGenerator;

architecture struct of InterruptGenerator is

```

```

begin
    -- IRQ(0) = receiver interrupt
    IRQ(0) <= RIE and (RDRF or OE);

    -- IRQ(1) = transmitter interrupt
    IRQ(1) <= TIE and TDRE;

end ;

```

trafficLighttopLevel

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

```

Entity trafficLighttopLevel IS

```

    PORT (
        GClock, GReset, SSCS, ProceedNextState : IN STD_LOGIC;
        MSC, SSC : IN STD_LOGIC_VECTOR(3 downto 0);
        MSTL, SSTL : OUT STD_LOGIC_VECTOR(2 downto 0);
        SegmentOut : OUT STD_LOGIC_VECTOR(13 downto 0)
    );
END trafficLighttopLevel;

```

architecture struct of trafficLighttopLevel IS

```

    COMPONENT BCD_to_7Segment
        PORT (
            A_in : IN STD_LOGIC_VECTOR(3 downto 0);
            seg : OUT STD_LOGIC_VECTOR(13 downto 0)
        );
    END COMPONENT;

    COMPONENT equality_Comparator_4bit
        PORT (
            A, B : IN STD_LOGIC_VECTOR(3 downto 0);
            isEqual : OUT STD_LOGIC
        );
    END COMPONENT;

    COMPONENT counter_4bit
        PORT (
            CLK : in STD_LOGIC;
            i_enable : in STD_LOGIC;

```

```

        i_hold : IN STD_LOGIC;
        i_reset : IN STD_LOGIC;
COUNT : out STD_LOGIC_VECTOR (3 downto 0)
    );
END COMPONENT;

COMPONENT mux_2to1_1bit
    PORT (
        d0 : IN STD_LOGIC;
d1 : IN STD_LOGIC;
sel : IN STD_LOGIC;
d_out : OUT STD_LOGIC
    );
END COMPONENT;

COMPONENT mux_2to1_2bit
    PORT (
        sel : IN STD_LOGIC;           -- Select input
d_in1 : IN STD_LOGIC_VECTOR(1 downto 0); -- 8-bit Data input 1
d_in2 : IN STD_LOGIC_VECTOR(1 downto 0); -- 8-bit Data input 2
Reset input
d_out : OUT STD_LOGIC_VECTOR(1 downto 0)
    );
END COMPONENT;

COMPONENT mux_4to1_1bit
    PORT (
        d_in : IN STD_LOGIC_VECTOR(3 downto 0);
        sel : IN STD_LOGIC_VECTOR(1 downto 0);
        en : IN STD_LOGIC;
        d_out : OUT STD_LOGIC
    );
END COMPONENT;

component d_FF
Port (
        i_d, i_en, i_reset : IN STD_LOGIC;
        i_clock : IN STD_LOGIC;
        o_q, o_qBar : OUT STD_LOGIC
    );
end component;

COMPONENT clk_div
    PORT (

```

```

        clock_25Mhz           : IN   STD_LOGIC;
        clock_1MHz            : OUT  STD_LOGIC;
        clock_100KHz          : OUT  STD_LOGIC;
        clock_10KHz           : OUT  STD_LOGIC;
        clock_1KHz            : OUT  STD_LOGIC;
        clock_100Hz           : OUT  STD_LOGIC;
        clock_10Hz            : OUT  STD_LOGIC;
        clock_1Hz             : OUT  STD_LOGIC;
    );
END COMPONENT;

signal TransitorySegmentOut : STD_LOGIC_VECTOR(13 downto 0);
signal MT, SST : STD_LOGIC_VECTOR(3 downto 0);
signal transitoryMSLT, transitoryMTTT, transitorySSTL, transitorySSTT, displayOnBCD,
displayOnBCD2 : STD_LOGIC_VECTOR(3 downto 0);
signal MSTLIsGreen, MSTLIsYellow, MSTLIsRed : STD_LOGIC;
signal SSTLIsGreen, SSTLIsYellow : STD_LOGIC;
signal STDtransitoryMSTL, STDtransitorySSTL, STDtransitoryMSTLDecider,
STDtransitorySSTLDecider : STD_LOGIC_VECTOR(1 downto 0);
signal BrakeNextState, MSLTHoldCond, MSTLReachedEnd, MTTTHoldCond,
SSTLHoldCond, SSTHoldCond : STD_LOGIC;
signal SSTLReachedEnd : STD_LOGIC;
signal SSTReachedEnd : STD_LOGIC;
signal MTRReachedEnd : STD_LOGIC;
begin
    MT <= "1100";
    SST <= "1000";

    STDtransitoryMSTL <= STDtransitoryMSTLDecider;
    STDtransitorySSTL <= STDtransitorySSTLDecider;
    MSLTHoldCond <= (NOT (ProceedNextState AND SSCS)) AND MSTLReachedEnd;
    MTTTHoldCond <= NOT ProceedNextState AND MTRReachedEnd;
    SSTLHoldCond <= NOT ProceedNextState AND SSTLReachedEnd;
    SSTHoldCond <= NOT ProceedNextState AND SSTReachedEnd;

    counter_4bit_MSLTimer : counter_4bit
        PORT MAP (
            CLK => GClock,
            i_enable => STDtransitoryMSTL(1),
            i_hold => MSLTHoldCond,
            i_reset => GReset,
            COUNT => transitoryMSLT
        );

```

```

equality_Comparator_4bit_MSLT_MSC : equality_Comparator_4bit
  PORT MAP (
    A => transitoryMSLT,
    B => MSC,
    isEqual => MSTLReachedEnd
  );

```

```

counter_4bit_MTT : counter_4bit
  PORT MAP (
    CLK => GClock,
    i_enable => STDtransitoryMSTL(0),
    i_hold => MTTTHoldCond,
    i_reset => GReset,
    COUNT => transitoryMTTT
  );

```

```

equality_Comparator_4bit_MTTTT_MT : equality_Comparator_4bit
  PORT MAP (
    A => transitoryMTTT,
    B => MT,
    isEqual => MTReachedEnd
  );

```

```

counter_4bit_SSTimer : counter_4bit
  PORT MAP (
    CLK => GClock,
    i_enable => STDtransitorySSTL(1),
    i_hold => SSTLHoldCond,
    i_reset => GReset,
    COUNT => transitorySSLT
  );

```

```

equality_Comparator_4bit_SSLT_SSC : equality_Comparator_4bit
  PORT MAP (
    A => transitorySSLT,
    B => SSC,
    isEqual => SSTLReachedEnd
  );

```

```

counter_4bit_SST : counter_4bit
  PORT MAP (
    CLK => GClock,
    i_enable => STDtransitorySSTL(0),

```

```

        i_hold => SSTHoldCond,
        i_reset => GReset,
        COUNT => transitorySSTT
    );

equality_Comparator_4bit_SSTT_SST : equality_Comparator_4bit
    PORT MAP (
        A => transitorySSTT,
        B => SST,
        isEqual => SSTReachedEnd
    );

MSTLIsGreen <= (((NOT SSCS OR NOT MSTLReachedEnd) AND
(STDtransitoryMSTL(1) AND NOT STDtransitoryMSTL(0))) OR
(SSTReachedEnd OR (NOT STDtransitoryMSTL(1) OR NOT STDtransitoryMSTL(0))))
AND NOT (MSTLIsYellow OR STDtransitorySSTL(1) OR STDtransitorySSTL(0)) AND
NOT SSTLIsGreen;

MSTLIsYellow <= (((SSCS AND MSTLReachedEnd) AND (STDtransitoryMSTL(1) AND
NOT STDtransitoryMSTL(0))) OR
(NOT MTRReachedEnd AND (NOT STDtransitoryMSTL(1) AND STDtransitoryMSTL(0))));

MSTLIsRed <= (MTRReachedEnd AND (NOT STDtransitoryMSTL(1) AND
STDtransitoryMSTL(0))) OR
((NOT SSTReachedEnd OR NOT SSTLReachedEnd) AND (NOT STDtransitoryMSTL(1)
AND NOT STDtransitoryMSTL(0)));

BrakeNextState <= GClock AND (ProceedNextState OR ((NOT
STDtransitorySSTLDecider(1) AND NOT STDtransitorySSTLDecider(0))
AND (NOT STDtransitoryMSTLDecider(1) AND NOT STDtransitoryMSTLDecider(0))));
d_FF_MSTL_High : d_FF
    PORT MAP (
        i_d => MSTLIsGreen,
        i_en => '1',
        i_reset => GReset,
        i_clock => BrakeNextState,
        o_q => STDtransitoryMSTLDecider(1),
        o_qBar => open
    );

d_FF_MSTL_Low : d_FF
    PORT MAP (
        i_d => MSTLIsYellow,
        i_en => '1',

```

```

        i_reset => GReset,
        i_clock => BrakeNextState,
        o_q => STDtransitoryMSTLDecider(0),
        o_qBar => open
    );

```

```

    SSTLIsGreen <= (MTReachedEnd AND (NOT STDtransitorySSTL(1) AND NOT
STDtransitorySSTL(0))) OR
    (NOT SSTLReachedEnd AND (STDtransitorySSTL(1) AND NOT
STDtransitorySSTL(0)));

```

```

    SSTLIsYellow <= (SSTLReachedEnd AND (STDtransitorySSTL(1) AND NOT
STDtransitorySSTL(0))) OR
    (NOT SSTReachedEnd AND (NOT STDtransitorySSTL(1) AND STDtransitorySSTL(0)));

```

```

d_FF_SSTL_High : d_FF
    PORT MAP (
        i_d => SSTLIsGreen,
        i_en => '1',
        i_reset => GReset,
        i_clock => BrakeNextState,
        o_q => STDtransitorySSTLDecider(1),
        o_qBar => open
    );

```

```

d_FF_SSTL_Low : d_FF
    PORT MAP (
        i_d => SSTLIsYellow,
        i_en => '1',
        i_reset => GReset,
        i_clock => BrakeNextState,
        o_q => STDtransitorySSTLDecider(0),
        o_qBar => open
    );

```

```

    SSTL(0) <= NOT STDtransitorySSTLDecider(1) AND NOT
STDtransitorySSTLDecider(0);
    SSTL(1) <= STDtransitorySSTLDecider(0);
    SSTL(2) <= STDtransitorySSTLDecider(1);
    MSTL(0) <= NOT STDtransitoryMSTLDecider(1) AND NOT
STDtransitoryMSTLDecider(0);
    MSTL(1) <= STDtransitoryMSTLDecider(0);
    MSTL(2) <= STDtransitoryMSTLDecider(1);

```

```
displayOnBCD <= transitoryMSLT OR transitoryMTTT OR transitorySSLT OR
transitorySSTT;
```

```
BCD_to_7Segment_inst : BCD_to_7Segment
  PORT MAP (
    A_in => displayOnBCD,
    seg => TransitorySegmentOut
  );
SegmentOut <= NOT TransitorySegmentOut;
```

```
end struct;
```

Mux_4to1_1bit

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
```

```
ENTITY mux_4to1_1bit IS
  PORT(
    d_in : IN STD_LOGIC_VECTOR(3 downto 0);
    sel : IN STD_LOGIC_VECTOR(1 downto 0);
    d_out : OUT STD_LOGIC
  );
END mux_4to1_1bit;
```

```
ARCHITECTURE structural OF mux_4to1_1bit IS
  SIGNAL andResult : STD_LOGIC;
```

```
BEGIN
  andResult <= (NOT(sel(0)) AND NOT(sel(1)) AND d_in(0))
    OR (NOT(sel(0)) AND sel(1) AND d_in(1))
    OR (sel(0) AND NOT(sel(1)) AND d_in(2))
    OR (sel(0) AND sel(1) AND d_in(3));

  d_out <= andResult;
```

```
end structural;
```

Mux_2to1_4bit

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
```

```

ENTITY mux_2to1_4bit IS
  PORT(
    sel   : IN STD_LOGIC;           -- Select input
    d_in1  : IN STD_LOGIC_VECTOR(3 downto 0);  -- 8-bit Data input 1
    d_in2  : IN STD_LOGIC_VECTOR(3 downto 0);  -- 8-bit Data input 2
    Reset input
    d_out  : OUT STD_LOGIC_VECTOR(3 downto 0)  -- 8-bit Data output
  );
END mux_2to1_4bit;

```

ARCHITECTURE structural OF mux_2to1_4bit IS

```

  SIGNAL not_sel : STD_LOGIC;           -- Signal for inverted select line
  SIGNAL and1    : STD_LOGIC_VECTOR(3 downto 0);  -- Intermediate AND output for
input 1
  SIGNAL and2    : STD_LOGIC_VECTOR(3 downto 0);  -- Intermediate AND output for
input 2
  SIGNAL temp_out: STD_LOGIC_VECTOR(3 downto 0);  -- Temporary output

```

```

BEGIN
  not_sel <= NOT(sel);                 -- Invert the select signal

```

```

  -- AND gates for each bit of d_in1
  and1(0) <= d_in1(0) AND not_sel;
  and1(1) <= d_in1(1) AND not_sel;
  and1(2) <= d_in1(2) AND not_sel;
  and1(3) <= d_in1(3) AND not_sel;

```

```

  -- AND gates for each bit of d_in2
  and2(0) <= d_in2(0) AND sel;
  and2(1) <= d_in2(1) AND sel;
  and2(2) <= d_in2(2) AND sel;
  and2(3) <= d_in2(3) AND sel;

```

```

  -- Final output OR gates
  temp_out(0) <= and1(0) OR and2(0);
  temp_out(1) <= and1(1) OR and2(1);
  temp_out(2) <= and1(2) OR and2(2);
  temp_out(3) <= and1(3) OR and2(3);

```

```

  d_out <= temp_out;

```

```
END structural;
```

Mux_2to1_2bit

```
LIBRARY ieee;
```

```
USE ieee.std_logic_1164.ALL;
```

```
ENTITY mux_2to1_2bit IS
```

```
  PORT(
```

```
    sel  : IN STD_LOGIC;           -- Select input
```

```
    d_in1 : IN STD_LOGIC_VECTOR(1 downto 0);  -- 8-bit Data input 1
```

```
    d_in2 : IN STD_LOGIC_VECTOR(1 downto 0);  -- 8-bit Data input 2      --
```

```
  Reset input
```

```
    d_out : OUT STD_LOGIC_VECTOR(1 downto 0)    -- 8-bit Data output
```

```
  );
```

```
END mux_2to1_2bit;
```

```
ARCHITECTURE structural OF mux_2to1_2bit IS
```

```
  SIGNAL not_sel : STD_LOGIC;           -- Signal for inverted select line
```

```
  SIGNAL and1 : STD_LOGIC_VECTOR(1 downto 0);  -- Intermediate AND output for  
input 1
```

```
  SIGNAL and2 : STD_LOGIC_VECTOR(1 downto 0);  -- Intermediate AND output for  
input 2
```

```
  SIGNAL temp_out: STD_LOGIC_VECTOR(1 downto 0);  -- Temporary output
```

```
BEGIN
```

```
  not_sel <= NOT(sel);                 -- Invert the select signal
```

```
  -- AND gates for each bit of d_in1
```

```
  and1(0) <= d_in1(0) AND not_sel;
```

```
  and1(1) <= d_in1(1) AND not_sel;
```

```
  -- AND gates for each bit of d_in2
```

```
  and2(0) <= d_in2(0) AND sel;
```

```
  and2(1) <= d_in2(1) AND sel;
```

```
  -- Final output OR gates
```

```
  temp_out(0) <= and1(0) OR and2(0);
```

```
  temp_out(1) <= and1(1) OR and2(1);
```

```
  d_out <= temp_out;
```

```
END structural;
```

Mux_2to1_1bit

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY mux_2to1_1bit IS
    PORT(
        d0 : IN STD_LOGIC;
        d1 : IN STD_LOGIC;
        sel : IN STD_LOGIC;
        d_out : OUT STD_LOGIC
    );
END mux_2to1_1bit;

ARCHITECTURE structural OF mux_2to1_1bit IS
BEGIN
    d_out <= ((NOT(sel) AND d0) OR (sel AND d1));
END structural;
```

equality_Comparator_4bit

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY equality_Comparator_4bit IS
    PORT (
        A, B : IN STD_LOGIC_VECTOR(3 downto 0);
        isEqual : OUT STD_LOGIC
    );
END equality_Comparator_4bit;

architecture basic of equality_Comparator_4bit IS
    SIGNAL equalcond, greatercond : STD_LOGIC;
begin
    equalcond <= NOT (A(3) XOR B(3)) AND NOT (A(2) XOR B(2)) AND NOT (A(1) XOR
B(1)) AND NOT (A(0) XOR B(0));
    greatercond <= (A(3) AND NOT B(3)) OR
        (NOT (A(3) XOR B(3)) AND A(2) AND NOT B(2)) OR
        (NOT (A(3) XOR B(3)) AND NOT (A(2) XOR B(2)) AND A(1) AND NOT B(1)) OR
        (NOT (A(3) XOR B(3)) AND NOT (A(2) XOR B(2)) AND NOT (A(1) XOR B(1)) AND
A(0) AND NOT B(0));

    isEqual <= equalcond OR greatercond;
end basic;
```

enabledSRLatch

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY enabledSRLatch IS
    PORT(
        i_set, i_reset : IN    STD_LOGIC;
        i_enable       : IN    STD_LOGIC;
        o_q, o_qBar    : OUT   STD_LOGIC);
END enabledSRLatch;

ARCHITECTURE rtl OF enabledSRLatch IS
    SIGNAL int_q, int_qBar : STD_LOGIC;
    SIGNAL int_sSignal, int_rSignal : STD_LOGIC;
BEGIN

    -- Concurrent Signal Assignment

    int_sSignal    <=    i_set nand i_enable;
    int_rSignal    <=    i_enable nand i_reset;
    int_q          <=    int_sSignal nand int_qBar;
    int_qBar       <=    int_q nand int_rSignal;

    -- Output Driver

    o_q            <=    int_q;
    o_qBar         <=    int_qBar;

END rtl;
```

dFF_8bit

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY dFF_8bit IS
    PORT(
        i_d        : IN  STD_LOGIC_VECTOR(7 downto 0); -- 8-bit input data
        i_en        : IN  STD_LOGIC;
        i_reset     : IN  STD_LOGIC;
        i_clock     : IN  STD_LOGIC;                    -- Clock input
        o_q         : OUT STD_LOGIC_VECTOR(7 downto 0); -- 8-bit output data
        o_qBar      : OUT STD_LOGIC_VECTOR(7 downto 0)  -- 8-bit inverted output data
    );
```

```
END dFF_8bit;
```

```
ARCHITECTURE rtl OF dFF_8bit IS
```

```
  COMPONENT d_FF
```

```
    PORT(
```

```
      i_d, i_en, i_reset    : IN STD_LOGIC;
```

```
      i_clock  : IN STD_LOGIC;
```

```
      o_q      : OUT STD_LOGIC;
```

```
      o_qBar   : OUT STD_LOGIC
```

```
    );
```

```
  END COMPONENT;
```

```
  SIGNAL q_internal  : STD_LOGIC_VECTOR(7 downto 0);
```

```
  SIGNAL qBar_internal: STD_LOGIC_VECTOR(7 downto 0);
```

```
BEGIN
```

```
  dff_inst0: d_FF
```

```
    PORT MAP (
```

```
      i_d    => i_d(0),A
```

```
      i_en    => i_en,
```

```
      i_reset => i_reset,
```

```
      i_clock => i_clock,
```

```
      o_q     => q_internal(0),
```

```
      o_qBar  => qBar_internal(0)
```

```
    );
```

```
  dff_inst1: d_FF
```

```
    PORT MAP (
```

```
      i_d    => i_d(1),
```

```
      i_en    => i_en,
```

```
      i_reset => i_reset,
```

```
      i_clock => i_clock,
```

```
      o_q     => q_internal(1),
```

```
      o_qBar  => qBar_internal(1)
```

```
    );
```

```
  dff_inst2: d_FF
```

```
    PORT MAP (
```

```
      i_d    => i_d(2),
```

```
      i_en    => i_en,
```

```
      i_reset => i_reset,
```

```
      i_clock => i_clock,
```

```
    o_q    => q_internal(2),
    o_qBar => qBar_internal(2)
);
```

```
dff_inst3: d_FF
PORT MAP (
    i_d    => i_d(3),
                                i_en    => i_en,
                                i_reset => i_reset,
    i_clock => i_clock,
    o_q    => q_internal(3),
    o_qBar => qBar_internal(3)
);
```

```
dff_inst4: d_FF
PORT MAP (
    i_d    => i_d(4),
                                i_en    => i_en,
                                i_reset => i_reset,
    i_clock => i_clock,
    o_q    => q_internal(4),
    o_qBar => qBar_internal(4)
);
```

```
dff_inst5: d_FF
PORT MAP (
    i_d    => i_d(5),
                                i_en    => i_en,
                                i_reset => i_reset,
    i_clock => i_clock,
    o_q    => q_internal(5),
    o_qBar => qBar_internal(5)
);
```

```
dff_inst6: d_FF
PORT MAP (
    i_d    => i_d(6),
                                i_en    => i_en,
                                i_reset => i_reset,
    i_clock => i_clock,
    o_q    => q_internal(6),
    o_qBar => qBar_internal(6)
);
```

```

dff_inst7: d_FF
  PORT MAP (
    i_d    => i_d(7),
                                i_en    => i_en,
                                i_reset => i_reset,
    i_clock => i_clock,
    o_q     => q_internal(7),
    o_qBar  => qBar_internal(7)
  );

```

```

-- Assign internal signals to outputs

```

```

o_q  <= q_internal;
o_qBar <= qBar_internal;

```

```

END rtl;

```

d_FF

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

```

```

ENTITY d_FF IS
  PORT(
    i_d, i_en, i_reset : IN STD_LOGIC;
    i_clock            : IN STD_LOGIC;
    o_q, o_qBar       : OUT STD_LOGIC
  );
END d_FF;

```

```

ARCHITECTURE behavioral OF d_FF IS
  SIGNAL int_q : STD_LOGIC;
BEGIN
  PROCESS(i_clock, i_reset)
  BEGIN
    -- Reset Logic
    IF i_reset = '1' THEN
      int_q <= '0';

    -- Latch data on clock edge, or reset if enable is low
    ELSIF rising_edge(i_clock) THEN
      IF i_en = '1' THEN
        int_q <= i_d;    -- Capture input if enabled
      ELSE
        int_q <= '0';    -- Set to 0 if not enabled
      END IF;
    END IF;
  END PROCESS;

```

```

        END IF;
    END IF;
END PROCESS;

```

```

-- Output Assignments
o_q <= int_q;
o_qBar <= NOT int_q;
END behavioral;

```

counter_4bit

```

-- counter_4bit.vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

```

```

entity counter_4bit is
    Port (
        CLK : in STD_LOGIC;
        i_enable : in STD_LOGIC;
            i_hold : IN STD_LOGIC;
            i_reset : IN STD_LOGIC;
        COUNT : out STD_LOGIC_VECTOR (3 downto 0)
    );
end counter_4bit;

```

architecture Structural of counter_4bit is

component d_FF

```

    Port (
        i_d : IN STD_LOGIC;
        i_en : IN STD_LOGIC;
        i_reset : IN STD_LOGIC;
        i_clock : IN STD_LOGIC;
        o_q, o_qBar : OUT STD_LOGIC
    );
end component;

```

component mux_2to1_4bit

```

    PORT (
        sel : IN STD_LOGIC;           -- Select input
        d_in1 : IN STD_LOGIC_VECTOR(3 downto 0); -- 8-bit Data input 1
        d_in2 : IN STD_LOGIC_VECTOR(3 downto 0); -- 8-bit Data input 2
        Reset input
        d_out : OUT STD_LOGIC_VECTOR(3 downto 0)
    );

```

```

END COMPONENT;

signal t_en : std_LOGIC;
signal Q, td, td_2, td_3 : STD_LOGIC_VECTOR(3 downto 0);
begin
  -- Flip-flop instances
  FF0: d_FF
    port map (
      i_d => td(0),
      i_en => i_enable,
      i_reset => i_reset,
      i_clock => CLK,
      o_q => Q(0),
      o_qBar => open
    );
  FF1: d_FF
    port map (
      i_d => td(1),
      i_en => i_enable,
      i_reset => i_reset,
      i_clock => CLK,
      o_q => Q(1),
      o_qBar => open
    );
  FF2: d_FF
    port map (
      i_d => td(2),
      i_en => i_enable,
      i_reset => i_reset,
      i_clock => CLK,
      o_q => Q(2),
      o_qBar => open
    );
  FF3: d_FF
    port map (
      i_d => td(3),
      i_en => i_enable,
      i_reset => i_reset,
      i_clock => CLK,
      o_q => Q(3),
      o_qBar => open
    );

  td_2(0) <= NOT Q(0);

```

```

td_2(1) <= (Q(0) XOR Q(1));

td_2(2) <= ((Q(0) AND Q(1)) XOR Q(2));

td_2(3) <= ((Q(0) AND Q(1) AND Q(2)) XOR Q(3));

mux_2to1_4bit_inst : mux_2to1_4bit
    PORT MAP (
        sel => i_hold,
        d_in1 => td_2,
        d_in2 => Q,
        d_out => td
    );

COUNT <= Q;

```

end Structural;

BCD_to_7Segment

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY BCD_to_7Segment IS
    PORT (
        A_in : IN STD_LOGIC_VECTOR(3 downto 0);
        seg : OUT STD_LOGIC_VECTOR(13 downto 0)
    );
END BCD_to_7Segment;

ARCHITECTURE Structural OF BCD_to_7Segment IS
    SIGNAL A, B, C, D : STD_LOGIC;
BEGIN
    A <= A_in(3);
    B <= A_in(2);
    C <= A_in(1);
    D <= A_in(0);

    -- a
    seg(0) <= (NOT B AND NOT C AND NOT D) OR
        (NOT A AND C) OR
        (A AND NOT C) OR
        (B AND D);

```

--b

seg(1) <= (NOT A AND NOT B) OR
(NOT C AND NOT D) OR
(NOT B AND D) OR
(A AND NOT C) OR
(NOT A AND C AND D) OR
(A AND B AND NOT D);

--c

seg(2) <= D OR (B AND C) OR (NOT A AND NOT C) OR (NOT B AND NOT C);

--d

seg(3) <= (A AND NOT C) OR
(NOT A AND C AND NOT D) OR
(B AND NOT C AND D) OR
(NOT A AND NOT B AND NOT D) OR
(A AND B AND D) OR
(NOT A AND NOT B AND C);

--e

seg(4) <= (A AND NOT C AND NOT D) OR
(NOT A AND C AND NOT D) OR
(NOT A AND NOT B AND NOT D);

--f

seg(5) <= (NOT A AND B AND NOT C) OR
(NOT B AND NOT C AND NOT D) OR
(A AND NOT B AND NOT C) OR
(A AND B AND C) OR
(NOT A AND B AND NOT D);

--g

seg(6) <= (A AND B) OR
(B AND NOT C) OR
(A AND NOT C) OR
(NOT A AND C AND NOT D) OR
(NOT A AND NOT B AND C);

seg(7) <= '0';

seg(8) <= (A AND B) OR (A AND C);

seg(9) <= (A AND B) OR (A AND C);

```
seg(10) <= '0';
seg(11) <= '0';
seg(12) <= '0';
seg(13) <= '0';
```

END Structural;

Clk_div

```
-----
-- Title      : Clock Divider Circuit
-- Project    : VHDL Example Programs
-----
-- File       : clk_div.vhd
-- Author     : Rami Abielmona <rabiemo@site.uottawa.ca>
-- Created    : 2004/10/07
-- Last modified : 2007/09/26
-----
-- Description : This file creates a clock divider circuit using a behavioral approach.
--              The code is extracted from "Rapid Prototyping Of Digital
Systems"
--              by James Hamblen et Michael Furman.
-----
-- Modification history :
-- 2004.10.07 R. Abielmona      Creation
-- 2007.09.26 R. Abielmona      Modified copyright notice
-----
-- This file is copyright material of Rami Abielmona, Ph.D., P.Eng., Chief Research
-- Scientist at Larus Technologies. Permission to make digital or hard copies of part
-- or all of this work for personal or classroom use is granted without fee
-- provided that copies are not made or distributed for profit or commercial
-- advantage and that copies bear this notice and the full citation of this work.
-- Prior permission is required to copy, republish, redistribute or post this work.
-- This notice is adapted from the ACM copyright notice.
-----
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_ARITH.all;
use IEEE.STD_LOGIC_UNSIGNED.all;
```

ENTITY clk_div IS

PORT

```

(
    clock_25Mhz          : IN  STD_LOGIC;
    clock_1MHz           : OUT  STD_LOGIC;
    clock_100KHz         : OUT  STD_LOGIC;
    clock_10KHz          : OUT  STD_LOGIC;
    clock_1KHz           : OUT  STD_LOGIC;
    clock_100Hz          : OUT  STD_LOGIC;
    clock_10Hz           : OUT  STD_LOGIC;
    clock_1Hz            : OUT  STD_LOGIC);

END clk_div;

ARCHITECTURE a OF clk_div IS

    SIGNAL    count_1Mhz: STD_LOGIC_VECTOR(4 DOWNT0 0);
    SIGNAL    count_100Khz, count_10Khz, count_1Khz : STD_LOGIC_VECTOR(2
DOWNT0 0);
    SIGNAL    count_100hz, count_10hz, count_1hz : STD_LOGIC_VECTOR(2
DOWNT0 0);
    SIGNAL    clock_1Mhz_int, clock_100Khz_int, clock_10Khz_int, clock_1Khz_int:
STD_LOGIC;
    SIGNAL    clock_100hz_int, clock_10Hz_int, clock_1Hz_int : STD_LOGIC;
BEGIN
    PROCESS
    BEGIN
-- Divide by 25
        WAIT UNTIL clock_25Mhz'EVENT and clock_25Mhz = '1';
        IF count_1Mhz < 24 THEN
            count_1Mhz <= count_1Mhz + 1;
        ELSE
            count_1Mhz <= "00000";
        END IF;
        IF count_1Mhz < 12 THEN
            clock_1Mhz_int <= '0';
        ELSE
            clock_1Mhz_int <= '1';
        END IF;

-- Ripple clocks are used in this code to save prescalar hardware
-- Sync all clock prescalar outputs back to master clock signal
        clock_1Mhz <= clock_1Mhz_int;
        clock_100Khz <= clock_100Khz_int;
        clock_10Khz <= clock_10Khz_int;
        clock_1Khz <= clock_1Khz_int;
    END PROCESS
END a;

```

```

        clock_100hz <= clock_100hz_int;
        clock_10hz <= clock_10hz_int;
        clock_1hz <= clock_1hz_int;
    END PROCESS;

-- Divide by 10
    PROCESS
    BEGIN
        WAIT UNTIL clock_1Mhz_int'EVENT and clock_1Mhz_int = '1';
        IF count_100Khz /= 4 THEN
            count_100Khz <= count_100Khz + 1;
        ELSE
            count_100khz <= "000";
            clock_100Khz_int <= NOT clock_100Khz_int;
        END IF;
    END PROCESS;

-- Divide by 10
    PROCESS
    BEGIN
        WAIT UNTIL clock_100Khz_int'EVENT and clock_100Khz_int = '1';
        IF count_10Khz /= 4 THEN
            count_10Khz <= count_10Khz + 1;
        ELSE
            count_10khz <= "000";
            clock_10Khz_int <= NOT clock_10Khz_int;
        END IF;
    END PROCESS;

-- Divide by 10
    PROCESS
    BEGIN
        WAIT UNTIL clock_10Khz_int'EVENT and clock_10Khz_int = '1';
        IF count_1Khz /= 4 THEN
            count_1Khz <= count_1Khz + 1;
        ELSE
            count_1khz <= "000";
            clock_1Khz_int <= NOT clock_1Khz_int;
        END IF;
    END PROCESS;

-- Divide by 10
    PROCESS
    BEGIN

```

```

        WAIT UNTIL clock_1Khz_int'EVENT and clock_1Khz_int = '1';
        IF count_100hz /= 4 THEN
            count_100hz <= count_100hz + 1;
        ELSE
            count_100hz <= "000";
            clock_100hz_int <= NOT clock_100hz_int;
        END IF;
    END PROCESS;

-- Divide by 10
    PROCESS
    BEGIN
        WAIT UNTIL clock_100hz_int'EVENT and clock_100hz_int = '1';
        IF count_10hz /= 4 THEN
            count_10hz <= count_10hz + 1;
        ELSE
            count_10hz <= "000";
            clock_10hz_int <= NOT clock_10hz_int;
        END IF;
    END PROCESS;

-- Divide by 10
    PROCESS
    BEGIN
        WAIT UNTIL clock_10hz_int'EVENT and clock_10hz_int = '1';
        IF count_1hz /= 4 THEN
            count_1hz <= count_1hz + 1;
        ELSE
            count_1hz <= "000";
            clock_1hz_int <= NOT clock_1hz_int;
        END IF;
    END PROCESS;

END a;

```

Debouncer

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY debouncer IS
    PORT(
        i_reset      : IN    STD_LOGIC;
        i_clock       : IN    STD_LOGIC;

```

```

        i_raw          : IN    STD_LOGIC;
        o_clean        : OUT   STD_LOGIC);
END debouncer;

```

ARCHITECTURE rtl OF debouncer IS

```

    SIGNAL con_vcc : STD_LOGIC;
    SIGNAL int_notQ1Output, int_q1Output, int_notD1Input, int_d1Input :STD_LOGIC;
    SIGNAL int_q2Output, int_notQ2Output, int_d2Input, int_debouncedRaw, int_feedback :
STD_LOGIC;

```

```

    COMPONENT d_FF
    PORT(
        i_d, i_en, i_reset          : IN    STD_LOGIC;
        i_clock                     : IN    STD_LOGIC;
        o_q, o_qBar                 : OUT   STD_LOGIC);
    END COMPONENT;

```

BEGIN

```

    con_vcc <= '1';
first: d_FF
    PORT MAP (i_reset => i_reset,
        i_d => int_notD1Input,
        i_en => con_vcc,
        i_clock => i_clock,
        o_q => int_q1Output,
        o_qBar => int_notQ1Output);

```

```

second: d_FF
    PORT MAP (i_reset => i_reset,
        i_d => int_d2Input,
        i_en => con_vcc,
        i_clock => i_clock,
        o_q => int_q2Output,
        o_qBar => int_notQ2Output);

```

```

-- Internal concurrent signal assignment
int_notD1Input <= not(int_d1Input);
int_d1Input <= i_raw nand int_feedback;
int_feedback <= int_q2Output or int_debouncedRaw;
int_d2Input <= int_notQ1Output and int_notQ2Output and i_raw;
int_debouncedRaw <= int_q2Output nor int_notQ1Output;

```

```

-- Output Concurrent Signal Assignment
o_clean <= int_debouncedRaw;

```

END rtl;

debuggableTrafficLightController

LIBRARY IEEE;

USE ieee.std_logic_1164.ALL;

ENTITY debuggableTrafficLightController IS

PORT (

GClock, GReset, SSCS, RxD, simClock : IN STD_LOGIC;

SW1, SW2 : IN STD_LOGIC_VECTOR(3 downto 0);

TxD : OUT STD_LOGIC;

MSTL, SSTL : OUT STD_LOGIC_VECTOR(2 downto 0);

SegmentOut : OUT STD_LOGIC_VECTOR(13 downto 0)

);

END debuggableTrafficLightController;

ARCHITECTURE struct OF debuggableTrafficLightController IS

COMPONENT trafficLighttopLevel

PORT (

GClock, GReset, SSCS, ProceedNextState : IN STD_LOGIC;

MSC, SSC : IN STD_LOGIC_VECTOR(3 downto 0);

MSTL, SSTL : OUT STD_LOGIC_VECTOR(2 downto 0);

SegmentOut : OUT STD_LOGIC_VECTOR(13 downto 0)

);

END COMPONENT;

COMPONENT clk_div

PORT (

clock_25Mhz : IN STD_LOGIC;

clock_1MHz : OUT STD_LOGIC;

clock_100KHz : OUT STD_LOGIC;

clock_10KHz : OUT STD_LOGIC;

clock_1KHz : OUT STD_LOGIC;

clock_100Hz : OUT STD_LOGIC;

clock_10Hz : OUT STD_LOGIC;

clock_1Hz : OUT STD_LOGIC

);

END COMPONENT;

COMPONENT debouncer

PORT (

i_reset : IN STD_LOGIC;

i_clock : IN STD_LOGIC;

```

        i_raw          : IN   STD_LOGIC;
        o_clean        : OUT  STD_LOGIC
    );
END COMPONENT;

COMPONENT mux_2to1_1bit
    PORT (
        d0 : IN STD_LOGIC;
d1 : IN STD_LOGIC;
sel : IN STD_LOGIC;
d_out : OUT STD_LOGIC
    );
END COMPONENT;

COMPONENT UART
    PORT (
        RxD, Load, GReset, GClock : IN STD_LOGIC;
        State_Information : IN STD_LOGIC_VECTOR(5 downto 0);
        TxD, CanProceedState : OUT STD_LOGIC
    );
END COMPONENT;

    signal ProceedNextState, dividedClock, transitoryClock, debouncedSSCS :
STD_LOGIC;
    SIGNAL State_Information : STD_LOGIC_VECTOR(5 downto 0);
    signal transitorySW1, transitorySW2 : STD_LOGIC_VECTOR(3 downto 0);
    signal transitoryMSTL, transitorySSTL : STD_LOGIC_VECTOR(2 downto 0);
begin

    -- hit the little debounce
    debouncer_SCSS : debouncer
        PORT MAP (
            i_reset => GReset,
            i_clock => GClock,
            i_raw => SSCS,
            o_clean => debouncedSSCS
        );

    debouncer_sw10 : debouncer
        PORT MAP (
            i_reset => GReset,
            i_clock => GClock,
            i_raw => SW1(0),
            o_clean => transitorySW1(0)

```

```
);
```

```
debouncer_sw11 : debouncer  
  PORT MAP (  
    i_reset => GReset,  
    i_clock => GClock,  
    i_raw => SW1(1),  
    o_clean => transitorySW1(1)  
  );
```

```
debouncer_sw12 : debouncer  
  PORT MAP (  
    i_reset => GReset,  
    i_clock => GClock,  
    i_raw => SW1(2),  
    o_clean => transitorySW1(2)  
  );
```

```
debouncer_sw13 : debouncer  
  PORT MAP (  
    i_reset => GReset,  
    i_clock => GClock,  
    i_raw => SW1(3),  
    o_clean => transitorySW1(3)  
  );
```

```
debouncer_sw20 : debouncer  
  PORT MAP (  
    i_reset => GReset,  
    i_clock => GClock,  
    i_raw => SW2(0),  
    o_clean => transitorySW2(0)  
  );
```

```
debouncer_sw21 : debouncer  
  PORT MAP (  
    i_reset => GReset,  
    i_clock => GClock,  
    i_raw => SW2(1),  
    o_clean => transitorySW2(1)  
  );
```

```
debouncer_sw22 : debouncer  
  PORT MAP (  
    i_reset => GReset,  
    i_clock => GClock,  
    i_raw => SW2(2),  
    o_clean => transitorySW2(2)  
  );
```

```

        i_reset => GReset,
        i_clock => GClock,
        i_raw => SW2(2),
        o_clean => transitorySW2(2)
    );

debouncer_sw23 : debouncer
    PORT MAP (
        i_reset => GReset,
        i_clock => GClock,
        i_raw => SW2(3),
        o_clean => transitorySW2(3)
    );

clk_div_inst : clk_div
    PORT MAP (
        clock_25Mhz => GClock,
        clock_1MHz => dividedClock,
        clock_100KHz => open,
        clock_10KHz => open,
        clock_1KHz => open,
        clock_100Hz => open,
        clock_10Hz => open,
        clock_1Hz => open
    );

--set to 1 when simulating
mux_2to1_1bit_chooseClock : mux_2to1_1bit
    PORT MAP (
        d0 => dividedClock,
        d1 => GClock,
        sel => simClock,
        d_out => transitoryClock
    );

trafficLighttopLevel_inst : trafficLighttopLevel
    PORT MAP (
        GClock => transitoryClock,
        GReset => GReset,
        SSCS => debouncedSSCS,
        ProceedNextState => ProceedNextState,
        MSC => transitorySW1,

```

```

        SSC => transitorySW2,
        MSTL => transitoryMSTL,
        SSTL => transitorySSTL,
        SegmentOut => SegmentOut
    );

    State_Information(5 downto 3) <= transitoryMSTL;
    State_Information(2 downto 0) <= transitorySSTL;

    MSTL <= transitoryMSTL;
    SSTL <= transitorySSTL;

    UART_inst : UART
        PORT MAP (
            RxD => RxD,
            Load => '1',
            GReset => GReset,
            GClock => transitoryClock,
            State_Information => State_Information,
            TxD => TxD,
            CanProceedState => ProceedNextState
        );

end struct;

UARTAddressDecode

library ieee;
use ieee.std_logic_1164.all;

entity UARTAddressDecode is
    port(
        Address : in STD_LOGIC_VECTOR(1 downto 0);
        RWbar   : in STD_LOGIC;
        DecodeOut : out STD_LOGIC_VECTOR(5 downto 0));
end UARTAddressDecode;

architecture rtl of UARTAddressDecode is
    SIGNAL notAddress : STD_LOGIC_VECTOR(1 downto 0);
    SIGNAL notRWbar : STD_LOGIC;
begin
    notAddress <= not Address;

```

```

notRWbar <= not RWbar;

DecodeOut(0)<= notAddress(1) AND notAddress(0) AND notRWbar;
DecodeOut(1)<= notAddress(1) AND notAddress(0) AND RWbar;
DecodeOut(2)<= notAddress(1) AND Address(0) AND notRWbar;
DecodeOut(3)<= notAddress(1) AND Address(0) AND RWbar;
DecodeOut(4)<= Address(1) AND notRWbar;
DecodeOut(5)<= Address(1) AND RWbar;
end architecture rtl;

```

BaudRateGen

```

library ieee;
use ieee.std_logic_1164.all;

```

ENTITY BaudRateGen IS

```

    PORT (
        GClock, GReset : IN STD_LOGIC;
        SEL : IN STD_LOGIC_VECTOR(2 downto 0);
        BClk, BClkx8 : OUT STD_LOGIC
    );
END BaudRateGen;

```

ARCHITECTURE basic of BaudRateGen IS

```

    COMPONENT counter_6bit
        PORT (
            CLK : in STD_LOGIC;
            i_enable : in STD_LOGIC;
            i_hold : IN STD_LOGIC;
            i_reset : IN STD_LOGIC;
            COUNT : out STD_LOGIC_VECTOR (5 downto 0)
        );
    END COMPONENT;

    COMPONENT counter_8bit
        PORT (
            CLK : in STD_LOGIC;
            i_enable : in STD_LOGIC;
            i_hold : IN STD_LOGIC;
            i_reset : IN STD_LOGIC;
            COUNT : out STD_LOGIC_VECTOR (7 downto 0)
        );
    END COMPONENT;

```

```

COMPONENT equality_Comparator_6bit
  PORT (
    A, B : IN STD_LOGIC_VECTOR(5 downto 0);
    isEqual : OUT STD_LOGIC
  );
END COMPONENT;

```

```

COMPONENT mux_2to1_1bit
  PORT (
    d0 : IN STD_LOGIC;
    d1 : IN STD_LOGIC;
    sel : IN STD_LOGIC;
    d_out : OUT STD_LOGIC
  );
END COMPONENT;

```

```

COMPONENT mux_8to1_1bit
  PORT (
    d_in : IN STD_LOGIC_VECTOR(7 downto 0); -- 8 inputs
    sel : IN STD_LOGIC_VECTOR(2 downto 0); -- 3-bit selector
    d_out : OUT STD_LOGIC -- Output
  );
END COMPONENT;

```

```

component d_FF
Port (
    i_d, i_en, i_reset : IN STD_LOGIC;
    i_clock : IN STD_LOGIC;
    o_q, o_qBar : OUT STD_LOGIC
);
end component;

```

```

SIGNAL fourty, clockVal : STD_LOGIC_VECTOR(5 downto 0);
SIGNAL newclk : STD_LOGIC_VECTOR(7 downto 0);
SIGNAL STDDiv41Clock, div41IsEqual, div41IsEqualGated, resetCheck : STD_LOGIC;
SIGNAL STDDiv41ClockValue : STD_LOGIC := '1';

begin

fourty <= "101000";
resetCheck <= GReset OR div41IsEqualGated;

counter_6bit_div41 : counter_6bit
  PORT MAP (
    CLK => GClock,

```

```

        i_enable => '1',
        i_hold => '0',
        i_reset => resetCheck,
        COUNT => clockVal
    );

equality_Comparator_6bit_check41 : equality_Comparator_6bit
    PORT MAP (
        A => clockVal,
        B => forty,
        isEqual => div41IsEqual
    );

d_FF_Gate_isEqual : d_FF
    PORT MAP (
        i_d => div41IsEqual,
        i_en => '1',
        i_reset => GReset,
        i_clock => GClock,
        o_q => div41IsEqualGated,
        o_qBar => open
    );

STDDiv41ClockValue <= ((STDDiv41Clock XOR div41IsEqualGated) AND NOT GReset)
OR GReset;

d_FF_Gate_clk : d_FF
    PORT MAP (
        i_d => STDDiv41ClockValue,
        i_en => '1',
        i_reset => '0',
        i_clock => GClock,
        o_q => STDDiv41Clock,
        o_qBar => open
    );

counter_8bit_inst : counter_8bit
    PORT MAP (
        CLK => STDDiv41Clock,
        i_enable => '1',
        i_hold => '0',
        i_reset => GReset,
        COUNT => newclk
    );

```

```

mux_8to1_1bit_inst : mux_8to1_1bit
    PORT MAP (
        d_in => newclk,
        sel => SEL,
        d_out => BClk
    );

BClkx8 <= newclk(0);

end basic;

```

parityGen

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY parityGen IS
    PORT(
        i_d    : IN    STD_LOGIC_VECTOR(6 downto 0);
        p_out   : OUT   STD_LOGIC);
END parityGen;

ARCHITECTURE rtl OF parityGen IS

BEGIN

    p_out <= i_d(0) XOR i_d(1) XOR i_d(2) XOR i_d(3) XOR i_d(4) XOR i_d(5) XOR i_d(6);

END rtl;A

```

Mux_2to1_6bit

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY mux_2to1_6bit IS
    PORT(
        sel    : IN STD_LOGIC;           -- Select input
        d_in1   : IN STD_LOGIC_VECTOR(5 downto 0); -- 8-bit Data input 1
        d_in2   : IN STD_LOGIC_VECTOR(5 downto 0); -- 8-bit Data input 2
        Reset input
        d_out   : OUT STD_LOGIC_VECTOR(5 downto 0) -- 8-bit Data output
    );

```

```
END mux_2to1_6bit;
```

```
ARCHITECTURE structural OF mux_2to1_6bit IS
```

```
    SIGNAL not_sel : STD_LOGIC;           -- Signal for inverted select line
    SIGNAL and1   : STD_LOGIC_VECTOR(5 downto 0); -- Intermediate AND output for
input 1
    SIGNAL and2   : STD_LOGIC_VECTOR(5 downto 0); -- Intermediate AND output for
input 2
    SIGNAL temp_out: STD_LOGIC_VECTOR(5 downto 0); -- Temporary output
```

```
BEGIN
```

```
    not_sel <= NOT(sel);                -- Invert the select signal
```

```
    -- AND gates for each bit of d_in1
    and1(0) <= d_in1(0) AND not_sel;
    and1(1) <= d_in1(1) AND not_sel;
    and1(2) <= d_in1(2) AND not_sel;
    and1(3) <= d_in1(3) AND not_sel;
    and1(4) <= d_in1(4) AND not_sel;
    and1(5) <= d_in1(5) AND not_sel;
```

```
    -- AND gates for each bit of d_in2
    and2(0) <= d_in2(0) AND sel;
    and2(1) <= d_in2(1) AND sel;
    and2(2) <= d_in2(2) AND sel;
    and2(3) <= d_in2(3) AND sel;
    and2(4) <= d_in2(4) AND sel;
    and2(5) <= d_in2(5) AND sel;
```

```
    -- Final output OR gates
    temp_out(0) <= and1(0) OR and2(0);
    temp_out(1) <= and1(1) OR and2(1);
    temp_out(2) <= and1(2) OR and2(2);
    temp_out(3) <= and1(3) OR and2(3);
    temp_out(4) <= and1(4) OR and2(4);
    temp_out(5) <= and1(5) OR and2(5);
```

```
    d_out <= temp_out;
```

```
END structural;
```

Counter_6bit

```
-- counter_4bit.vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity counter_6bit is
  Port (
    CLK : in STD_LOGIC;
    i_enable : in STD_LOGIC;
        i_hold : IN STD_LOGIC;
        i_reset : IN STD_LOGIC;
    COUNT : out STD_LOGIC_VECTOR (5 downto 0)
  );
end counter_6bit;

architecture Structural of counter_6bit is
  component d_FF
    Port (
      i_d : IN STD_LOGIC;
      i_en : IN STD_LOGIC;
      i_reset : IN STD_LOGIC;
      i_clock : IN STD_LOGIC;
      o_q, o_qBar : OUT STD_LOGIC
    );
  end component;

  component mux_2to1_6bit
    PORT (
      sel : IN STD_LOGIC;           -- Select input
      d_in1 : IN STD_LOGIC_VECTOR(5 downto 0); -- 8-bit Data input 1
      d_in2 : IN STD_LOGIC_VECTOR(5 downto 0); -- 8-bit Data input 2
      Reset input
      d_out : OUT STD_LOGIC_VECTOR(5 downto 0)
    );
  END COMPONENT;

  signal t_en : std_LOGIC;
  signal Q, td, td_2, td_3 : STD_LOGIC_VECTOR(5 downto 0);
begin
  -- Flip-flop instances
  FF0: d_FF
    port map (
      i_d => td(0),
```

```

        i_en => i_enable,
        i_reset => i_reset,
        i_clock => CLK,
        o_q => Q(0),
        o_qBar => open
    );
FF1: d_FF
    port map (
        i_d => td(1),
        i_en => i_enable,
        i_reset => i_reset,
        i_clock => CLK,
        o_q => Q(1),
        o_qBar => open
    );
FF2: d_FF
    port map (
        i_d => td(2),
        i_en => i_enable,
        i_reset => i_reset,
        i_clock => CLK,
        o_q => Q(2),
        o_qBar => open
    );
FF3: d_FF
    port map (
        i_d => td(3),
        i_en => i_enable,
        i_reset => i_reset,
        i_clock => CLK,
        o_q => Q(3),
        o_qBar => open
    );
FF4: d_FF
    port map (
        i_d => td(4),
        i_en => i_enable,
        i_reset => i_reset,
        i_clock => CLK,
        o_q => Q(4),
        o_qBar => open
    );
FF5: d_FF
    port map (

```

```

        i_d => td(5),
        i_en => i_enable,
        i_reset => i_reset,
        i_clock => CLK,
        o_q => Q(5),
        o_qBar => open
    );

    td_2(0) <= NOT Q(0);

    td_2(1) <= (Q(0) XOR Q(1));

    td_2(2) <= ((Q(0) AND Q(1)) XOR Q(2));

    td_2(3) <= ((Q(0) AND Q(1) AND Q(2)) XOR Q(3));

    td_2(4) <= ((Q(0) AND Q(1) AND Q(2)) AND Q(3)) XOR Q(4);

    td_2(5) <= ((Q(0) AND Q(1) AND Q(2)) AND Q(3) AND Q(4)) XOR Q(5);

    mux_2to1_6bit_inst : mux_2to1_6bit
        PORT MAP (
            sel => i_hold,
            d_in1 => td_2,
            d_in2 => Q,
            d_out => td
        );

    COUNT <= Q;

end Structural;

```

equality_Comparator_6bit

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY equality_Comparator_6bit IS
    PORT (
        A, B : IN STD_LOGIC_VECTOR(5 downto 0);
        isEqual : OUT STD_LOGIC
    );
END equality_Comparator_6bit;

```

architecture basic of equality_Comparator_6bit IS

SIGNAL equalcond : STD_LOGIC;

begin

equalcond <= NOT (A(5) XOR B(5)) AND
NOT (A(4) XOR B(4)) AND
NOT (A(3) XOR B(3)) AND
NOT (A(2) XOR B(2)) AND
NOT (A(1) XOR B(1)) AND
NOT (A(0) XOR B(0));

isEqual <= equalcond;

end basic;

Counter_8bit

-- counter_4bit.vhdl

library IEEE;

use IEEE.STD_LOGIC_1164.ALL;

entity counter_8bit is

Port (

CLK : in STD_LOGIC;

i_enable : in STD_LOGIC;

i_hold : IN STD_LOGIC;

i_reset : IN STD_LOGIC;

COUNT : out STD_LOGIC_VECTOR (7 downto 0)

);

end counter_8bit;

architecture Structural of counter_8bit is

component d_FF

Port (

i_d : IN STD_LOGIC;

i_en : IN STD_LOGIC;

i_reset : IN STD_LOGIC;

i_clock : IN STD_LOGIC;

o_q, o_qBar : OUT STD_LOGIC

);

end component;

component mux_2to1_8bit

PORT (

sel : IN STD_LOGIC;

-- Select input

```

        d_in1 : IN STD_LOGIC_VECTOR(7 downto 0);    -- 8-bit Data input 1
        d_in2 : IN STD_LOGIC_VECTOR(7 downto 0);    -- 8-bit Data input 2    --
Reset input
        d_out : OUT STD_LOGIC_VECTOR(7 downto 0)
    );
    END COMPONENT;

    signal t_en : std_LOGIC;
    signal Q, td, td_2, td_3 : STD_LOGIC_VECTOR(7 downto 0);
begin
    -- Flip-flop instances
    FF0: d_FF
        port map (
            i_d => td(0),
            i_en => i_enable,
            i_reset => i_reset,
            i_clock => CLK,
            o_q => Q(0),
            o_qBar => open
        );
    FF1: d_FF
        port map (
            i_d => td(1),
            i_en => i_enable,
            i_reset => i_reset,
            i_clock => CLK,
            o_q => Q(1),
            o_qBar => open
        );
    FF2: d_FF
        port map (
            i_d => td(2),
            i_en => i_enable,
            i_reset => i_reset,
            i_clock => CLK,
            o_q => Q(2),
            o_qBar => open
        );
    FF3: d_FF
        port map (
            i_d => td(3),
            i_en => i_enable,
            i_reset => i_reset,
            i_clock => CLK,

```

```

        o_q => Q(3),
        o_qBar => open
    );
FF4: d_FF
    port map (
        i_d => td(4),
        i_en => i_enable,
        i_reset => i_reset,
        i_clock => CLK,
        o_q => Q(4),
        o_qBar => open
    );
FF5: d_FF
    port map (
        i_d => td(5),
        i_en => i_enable,
        i_reset => i_reset,
        i_clock => CLK,
        o_q => Q(5),
        o_qBar => open
    );
FF6: d_FF
    port map (
        i_d => td(6),
        i_en => i_enable,
        i_reset => i_reset,
        i_clock => CLK,
        o_q => Q(6),
        o_qBar => open
    );
FF7: d_FF
    port map (
        i_d => td(7),
        i_en => i_enable,
        i_reset => i_reset,
        i_clock => CLK,
        o_q => Q(7),
        o_qBar => open
    );

```

```
td_2(0) <= NOT Q(0);
```

```
td_2(1) <= (Q(0) XOR Q(1));
```

```

    td_2(2) <= ((Q(0) AND Q(1)) XOR Q(2));

    td_2(3) <= ((Q(0) AND Q(1) AND Q(2)) XOR Q(3));

    td_2(4) <= ((Q(0) AND Q(1) AND Q(2)) AND Q(3)) XOR Q(4);

    td_2(5) <= ((Q(0) AND Q(1) AND Q(2)) AND Q(3) AND Q(4)) XOR Q(5);

    td_2(6) <= ((Q(0) AND Q(1) AND Q(2)) AND Q(3) AND Q(4) AND Q(5)) XOR Q(6);

    td_2(7) <= ((Q(0) AND Q(1) AND Q(2)) AND Q(3) AND Q(4) AND Q(5) AND Q(6)) XOR
Q(7);

    mux_2to1_8bit_inst : mux_2to1_8bit
        PORT MAP (
            sel => i_hold,
            d_in1 => td_2,
            d_in2 => Q,
            d_out => td
        );

    COUNT <= Q;

end Structural;

```

Mux_2to1_8bit

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY mux_2to1_8bit IS
    PORT(
        sel    : IN STD_LOGIC;           -- Select input
        d_in1   : IN STD_LOGIC_VECTOR(7 downto 0); -- 8-bit Data input 1
        d_in2   : IN STD_LOGIC_VECTOR(7 downto 0); -- 8-bit Data input 2
        Reset input
        d_out    : OUT STD_LOGIC_VECTOR(7 downto 0) -- 8-bit Data output
    );
END mux_2to1_8bit;

```

ARCHITECTURE structural OF mux_2to1_8bit IS

```

    SIGNAL not_sel : STD_LOGIC;           -- Signal for inverted select line
    SIGNAL and1   : STD_LOGIC_VECTOR(7 downto 0); -- Intermediate AND output for
input 1
    SIGNAL and2   : STD_LOGIC_VECTOR(7 downto 0); -- Intermediate AND output for
input 2
    SIGNAL temp_out: STD_LOGIC_VECTOR(7 downto 0); -- Temporary output

BEGIN
    not_sel <= NOT(sel);                  -- Invert the select signal

    -- AND gates for each bit of d_in1
    and1(0) <= d_in1(0) AND not_sel;
    and1(1) <= d_in1(1) AND not_sel;
    and1(2) <= d_in1(2) AND not_sel;
    and1(3) <= d_in1(3) AND not_sel;
    and1(4) <= d_in1(4) AND not_sel;
    and1(5) <= d_in1(5) AND not_sel;
    and1(6) <= d_in1(6) AND not_sel;
    and1(7) <= d_in1(7) AND not_sel;

    -- AND gates for each bit of d_in2
    and2(0) <= d_in2(0) AND sel;
    and2(1) <= d_in2(1) AND sel;
    and2(2) <= d_in2(2) AND sel;
    and2(3) <= d_in2(3) AND sel;
    and2(4) <= d_in2(4) AND sel;
    and2(5) <= d_in2(5) AND sel;
    and2(6) <= d_in2(6) AND sel;
    and2(7) <= d_in2(7) AND sel;

    -- Final output OR gates
    temp_out(0) <= and1(0) OR and2(0);
    temp_out(1) <= and1(1) OR and2(1);
    temp_out(2) <= and1(2) OR and2(2);
    temp_out(3) <= and1(3) OR and2(3);
    temp_out(4) <= and1(4) OR and2(4);
    temp_out(5) <= and1(5) OR and2(5);
    temp_out(6) <= and1(6) OR and2(5);
    temp_out(7) <= and1(7) OR and2(5);

    d_out <= temp_out;

END structural;

```

Mux_8to1_1bit

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY mux_8to1_1bit IS
    PORT(
        d_in : IN STD_LOGIC_VECTOR(7 downto 0); -- 8 inputs
        sel : IN STD_LOGIC_VECTOR(2 downto 0); -- 3-bit selector
        d_out : OUT STD_LOGIC -- Output
    );
END mux_8to1_1bit;

ARCHITECTURE structural OF mux_8to1_1bit IS
    SIGNAL sel_not : STD_LOGIC_VECTOR(2 downto 0);
    SIGNAL and_terms : STD_LOGIC_VECTOR(7 downto 0);
BEGIN
    sel_not(0) <= NOT sel(0);
    sel_not(1) <= NOT sel(1);
    sel_not(2) <= NOT sel(2);

    and_terms(0) <= d_in(0) AND sel_not(2) AND sel_not(1) AND sel_not(0);
    and_terms(1) <= d_in(1) AND sel_not(2) AND sel_not(1) AND sel(0);
    and_terms(2) <= d_in(2) AND sel_not(2) AND sel(1) AND sel_not(0);
    and_terms(3) <= d_in(3) AND sel_not(2) AND sel(1) AND sel(0);
    and_terms(4) <= d_in(4) AND sel(2) AND sel_not(1) AND sel_not(0);
    and_terms(5) <= d_in(5) AND sel(2) AND sel_not(1) AND sel(0);
    and_terms(6) <= d_in(6) AND sel(2) AND sel(1) AND sel_not(0);
    and_terms(7) <= d_in(7) AND sel(2) AND sel(1) AND sel(0);

    d_out <= and_terms(0) OR and_terms(1) OR and_terms(2) OR and_terms(3) OR
        and_terms(4) OR and_terms(5) OR and_terms(6) OR and_terms(7);
END structural;
```

UARTTransmitter

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY UARTTransmitter IS
    PORT (
        loadTDR, GReset, GClock, BaudRate, shiftLoad : IN STD_LOGIC;
        D_In : IN STD_LOGIC_VECTOR(6 downto 0);
```

```

        TxD, TDRE, countEqualTenOut : OUT STD_LOGIC
    );
END UARTTransmitter;

```

ARCHITECTURE basic of UARTTransmitter IS

```

COMPONENT dFF_8bit
    PORT (
        i_d      : IN  STD_LOGIC_VECTOR(7 downto 0); -- 8-bit input data
        i_en      : IN  STD_LOGIC;
        i_reset   : IN  STD_LOGIC;
        i_clock   : IN  STD_LOGIC;                    -- Clock input
        o_q       : OUT STD_LOGIC_VECTOR(7 downto 0); -- 8-bit output data
        o_qBar    : OUT STD_LOGIC_VECTOR(7 downto 0)
    );
END COMPONENT;

```

```

COMPONENT ParallelInSerialOut_8Bit
    PORT (
        i_d      : IN  STD_LOGIC_VECTOR(9 downto 0);
        i_reset  : IN  STD_LOGIC;
        i_enable  : IN  STD_LOGIC;
        i_shift  : IN  STD_LOGIC;
        i_shift_Load : IN  STD_LOGIC;
        i_clock   : IN  STD_LOGIC;
        s_out    : OUT  STD_LOGIC
    );
END COMPONENT;

```

```

COMPONENT parityGen
    PORT (
        i_d      : IN  STD_LOGIC_VECTOR(6 downto 0);
        p_out    : OUT  STD_LOGIC
    );
END COMPONENT;

```

```

COMPONENT mux_2to1_1bit
    PORT (
        d0 : IN STD_LOGIC;
        d1 : IN STD_LOGIC;
        sel : IN STD_LOGIC;
        d_out : OUT STD_LOGIC
    );
END COMPONENT;

```

```

        COMPONENT counter_4bit
            PORT (
                CLK : in STD_LOGIC;
i_enable : in STD_LOGIC;
                i_hold : IN STD_LOGIC;
                i_reset : IN STD_LOGIC;
COUNT : out STD_LOGIC_VECTOR (3 downto 0)
            );
        END COMPONENT;

        COMPONENT equality_Comparator_4bit
            PORT (
                A, B : IN STD_LOGIC_VECTOR(3 downto 0);
                isEqual : OUT STD_LOGIC
            );
        END COMPONENT;

        signal transitoryI_D : STD_LOGIC_VECTOR(9 downto 0);
        signal d_indff, d_indffLatched : STD_LOGIC_VECTOR(7 downto 0);
        signal transitorySerialOut, enableCount, countEqualTen : STD_LOGIC;
        signal countValue : STD_LOGIC_VECTOR(3 downto 0);
begin

        transitoryI_D(9) <= '1';
        transitoryI_D(0) <= '0';

        d_indff(6 downto 0) <= D_In;
        d_indff(7) <= '0';

        dFF_8bitID : dFF_8bit
            PORT MAP (
                i_d => d_indff,
                i_en => loadTDR,
                i_reset => GReset,
                i_clock => GClock,
                o_q => d_indffLatched,
                o_qBar => open
            );

        transitoryI_D(7 downto 1) <= d_indffLatched(6 downto 0);

        parityGen_check : parityGen

```

```

PORT MAP (
    i_d => transitoryI_D(7 downto 1),
    p_out => transitoryI_D(8)
);

```

ParallelInSerialOut_8BitTXD : ParallelInSerialOut_8Bit

```

PORT MAP (
    i_d => transitoryI_D,
    i_reset => GReset,
    i_enable => '1',
    i_shift => '1',
    i_shift_Load => shiftLoad,
    i_clock => BaudRate,
    s_out => transitorySerialOut
);

```

mux_2to1_1bit_selout : mux_2to1_1bit

```

PORT MAP (
    d0 => transitorySerialOut,
    d1 => '1',
    sel => shiftLoad,
    d_out => TxD
);

```

enableCount <= not countEqualTen;

counter_4bit_count_10 : counter_4bit

```

PORT MAP (
    CLK => BaudRate,
    i_enable => enableCount,
    i_hold => '0',
    i_reset => GReset,
    COUNT => countValue
);

```

equality_Comparator_4bit_donesend : equality_Comparator_4bit

```

PORT MAP (
    A => countValue,
    B => "1010",
    isEqual => countEqualTen
);

```

countEqualTenOut <= countEqualTen;

TDRE <= countEqualTen OR

```

        (NOT (countValue(3) XOR '0') AND
          NOT (countValue(2) XOR '0') AND
          NOT (countValue(1) XOR '0') AND
          NOT (countValue(0) XOR '0'));

```

```

end basic;

```

UARTReciever

```

LIBRARY ieee;

```

```

USE ieee.std_logic_1164.ALL;

```

```

ENTITY UARTReciever IS

```

```

    PORT (
        RxD, GReset, BClkx8, GClock : IN STD_LOGIC;
        RecieveOut : OUT STD_LOGIC_VECTOR(7 downto 0);
        RDRF : OUT STD_LOGIC
    );

```

```

END UARTReciever;

```

```

ARCHITECTURE basic of UARTReciever IS

```

```

    COMPONENT counter_4bit
        PORT (
            CLK : in STD_LOGIC;
i_enable : in STD_LOGIC;
            i_hold : IN STD_LOGIC;
            i_reset : IN STD_LOGIC;
COUNT : out STD_LOGIC_VECTOR (3 downto 0)
        );
    END COMPONENT;

```

```

    COMPONENT equality_Comparator_4bit
        PORT (
            A, B : IN STD_LOGIC_VECTOR(3 downto 0);
            isEqual : OUT STD_LOGIC
        );
    END COMPONENT;

```

```

    COMPONENT SerialInParallelOut_8Bit
        PORT (
            i_reset : IN STD_LOGIC;
            i_enable : IN STD_LOGIC;
            i_shift : IN STD_LOGIC;

```

```

        i_clock : IN    STD_LOGIC;
        s_out   : OUT  STD_LOGIC_VECTOR(7 downto 0)
    );
END COMPONENT;

COMPONENT dFF_8bit
    PORT (
        i_d      : IN  STD_LOGIC_VECTOR(7 downto 0); -- 8-bit input data
        i_en      : IN  STD_LOGIC;
        i_reset   : IN  STD_LOGIC;
        i_clock   : IN  STD_LOGIC;                  -- Clock input
        o_q       : OUT STD_LOGIC_VECTOR(7 downto 0); -- 8-bit output data
        o_qBar    : OUT STD_LOGIC_VECTOR(7 downto 0)
    );
END COMPONENT;

COMPONENT d_FF
PORT(
    i_d, i_en, i_reset   : IN  STD_LOGIC;
    i_clock : IN  STD_LOGIC;
    o_q     : OUT STD_LOGIC;
    o_qBar  : OUT STD_LOGIC
);
END COMPONENT;

    SIGNAL countFourVal, countEightVal, numBitsCollected : STD_LOGIC_VECTOR(3
downto 0);
    SIGNAL transitoryPRecieve : STD_LOGIC_VECTOR(7 downto 0);
    SIGNAL isInMiddle, isAtEight, startSample, isStartBit, Latch8, Latch4, captureBit,
finishedCapture, finishedCaptureLatch, resetbitCounter : STD_LOGIC;
begin
    counter_4bit_countmiddle : counter_4bit
        PORT MAP (
            CLK => BClkx8,
            i_enable => isInMiddle,
            i_hold => '0',
            i_reset => GReset,
            COUNT => countFourVal
        );
    counter_4bit_counteight : counter_4bit
        PORT MAP (
            CLK => BClkx8,
            i_enable => Latch8,
            i_hold => '0',

```

```

        i_reset => GReset,
        COUNT => countEightVal
    );

    isInMiddle <= NOT isStartBit AND NOT startSample AND NOT (NOT (countFourVal(3)
XOR '0')
    AND NOT (countFourVal(2) XOR '0') AND NOT (countFourVal(1) XOR '1') AND NOT
(countFourVal(0) XOR '1'));
    isAtEight <= (NOT isInMiddle AND isStartBit) OR (startSample AND NOT (NOT
(countEightVal(3) XOR '0')
    AND NOT (countEightVal(2) XOR '1') AND NOT (countEightVal(1) XOR '1') AND NOT
(countEightVal(0) XOR '0')));
    d_FF_latch8 : d_FF
    PORT MAP (
        i_d => isAtEight,
        i_en => '1',
        i_reset => GReset,
        i_clock => BClkx8,
        o_q => Latch8,
        o_qBar => open
    );
    d_FF_latch4 : d_FF
    PORT MAP (
        i_d => isInMiddle,
        i_en => '1',
        i_reset => GReset,
        i_clock => BClkx8,
        o_q => Latch4,
        o_qBar => open
    );
    isStartBit <= NOT RxD AND NOT Latch4 AND NOT (numBitsCollected(3) XOR '0') AND
NOT (numBitsCollected(2) XOR '0') AND NOT (numBitsCollected(1) XOR '0') AND NOT
(numBitsCollected(0) XOR '0');
    equality_Comparator_4bit_startsmpl : equality_Comparator_4bit
    PORT MAP (
        A => numBitsCollected,
        B => "0001",
        isEqual => startSample
    );

    equality_Comparator_4bit_finishsmpl : equality_Comparator_4bit
    PORT MAP (
        A => numBitsCollected,
        B => "1001",

```

```

        isEqual => finishedCapture
    );
d_FF_finishedCap : d_FF
    PORT MAP (
        i_d => finishedCapture,
        i_en => '1',
        i_reset => GReset,
        i_clock => BClkx8,
        o_q => finishedCaptureLatch,
        o_qBar => open
    );
resetbitCounter <= finishedCaptureLatch OR GReset;
captureBit <= (NOT Latch4 AND Latch8);
counter_4bit_countRecieved : counter_4bit
    PORT MAP (
        CLK => captureBit,
        i_enable => '1',
        i_hold => '0',
        i_reset => resetbitCounter,
        COUNT => numBitsCollected
    );

SerialInParallelOut_8Bitinst : SerialInParallelOut_8Bit
    PORT MAP (
        i_reset => GReset,
        i_enable => '1',
        i_shift => RxD,
        i_clock => Latch8,
        s_out => transitoryPRecieve
    );

dFF_8bit_RDR : dFF_8bit
    PORT MAP (
        i_d => transitoryPRecieve,
        i_en => '1',
        i_reset => GReset,
        i_clock => BClkx8,
        o_q => RecieveOut,
        o_qBar => open
    );

RDRF <= finishedCapture;

```

end basic;

